

BlueDesk

Sistema web de gerenciamento de chamados de suporte técnico

Documento de estrutura do projeto: stack, arquitetura, banco de dados, mockups de tela, versionamento e roteiro de desenvolvimento.

1. Visão geral e stack tecnológica

O BlueDesk é um sistema de chamados de suporte técnico. O projeto foi ajustado para um primeiro contato simultâneo com SQL, API e JavaScript, priorizando ferramentas mais explícitas em vez das mais automatizadas, o que evita gerar dependência de mágica de framework.

Stack recomendada

Camada	Ferramenta	Motivo
Back-end	Python + Flask	Rotas explícitas, fácil de debugar; boa base antes de migrar para FastAPI
Banco de dados	SQLite + módulo sqlite3 nativo	Sem servidor para configurar; SQL escrito à mão, sem ORM escondendo a
Front-end	HTML, CSS e JavaScript puro	Sem framework: o foco fica em entender requisição/resposta com a API
Arquitetura	API REST (JSON sobre HTTP)	Padrão de mercado; front e back desacoplados
Versionamento	Git + GitHub (SSH)	Histórico de código e backup remoto

Evolução natural: depois que SQL e o ciclo de API estiverem confortáveis, migrar sqlite3 → SQLAlchemy + Alembic e Flask → FastAPI passa a fazer sentido, pois nesse ponto as abstrações somam produtividade em vez de esconder conceitos ainda não aprendidos.

2. Estrutura de pastas do projeto

Organização adaptada à fase de aprendizado atual (sqlite3 puro, sem ORM/migrations):

```

bluedesk/
├── backend/
│   ├── app.py           # cria o Flask app, registra as rotas
│   ├── db.py            # conexão sqlite3 + funções de acesso ao banco
│   ├── schema.sql       # script de criação das tabelas (rodado 1x)
│   └── core/
│       ├── config.py    # variáveis de ambiente
│       ├── security.py  # hash de senha, JWT, get_current_user
│       └── routes/
│           ├── auth.py  # login / registro
│           ├── chamados.py # POST /chamados, GET /chamados/me
│           ├── usuarios.py # GET/PUT /usuarios/me
│           └── dashboard.py # contadores do dashboard
│       ├── requirements.txt
│       ├── bluedesk.db    # arquivo do banco SQLite (gerado)
│       ├── .env
│       └──
└── frontend/
    ├── index.html        # tela de login
    ├── pages/
    │   ├── dashboard.html
    │   ├── chamados.html # "Meus chamados"
    │   ├── novo-chamado.html
    │   └── perfil.html
    ├── assets/
    │   ├── css/
    │   │   └── style.css
    │   ├── js/
    │   │   ├── api.js      # fetch com token, usado por todas as páginas
    │   │   ├── auth.js     # login/logout
    │   │   ├── dashboard.js
    │   │   ├── chamados.js  # listar + filtrar
    │   │   ├── novo-chamado.js # criar
    │   │   └── perfil.js    # ler + atualizar
    │   └── img/

```

Regra prática: 1 página HTML corresponde a 1 arquivo JS. Isso evita um arquivo único fazendo tudo, e cada página importa apenas o api.js compartilhado.

3. Banco de dados

Tabela	Campos principais
usuarios	id, nome, email (único), senha_hash, papel (admin agente cliente), criado_em
chamados	id, titulo, descricao, solicitante_id (FK), responsavel_id (FK, opcional), categoria_id (FK), status (aberto andamen
categorias	id, nome (Hardware, Rede, Sistema, Acesso...)
comentarios	id, chamado_id (FK), usuario_id (FK), mensagem, criado_em
anexos	id, chamado_id (FK), caminho_arquivo, enviado_por (FK), criado_em
historico_status	id, chamado_id (FK), status_anterior, status_novo, alterado_por (FK), alterado_em

Script inicial (schema.sql)

```
CREATE TABLE usuarios (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nome TEXT NOT NULL,  
  email TEXT UNIQUE NOT NULL,  
  senha_hash TEXT NOT NULL,  
  papel TEXT NOT NULL DEFAULT 'cliente',  
  criado_em TEXT DEFAULT CURRENT_TIMESTAMP  
);  
  
CREATE TABLE categorias (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  nome TEXT NOT NULL  
);  
  
CREATE TABLE chamados (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  titulo TEXT NOT NULL,  
  descricao TEXT NOT NULL,  
  solicitante_id INTEGER NOT NULL REFERENCES usuarios(id),  
  responsavel_id INTEGER REFERENCES usuarios(id),  
  categoria_id INTEGER REFERENCES categorias(id),  
  status TEXT NOT NULL DEFAULT 'aberto',  
  prioridade TEXT NOT NULL DEFAULT 'media',  
  criado_em TEXT DEFAULT CURRENT_TIMESTAMP,  
  atualizado_em TEXT,  
  resolvido_em TEXT  
);  
  
CREATE TABLE comentarios (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  chamado_id INTEGER NOT NULL REFERENCES chamados(id),  
  usuario_id INTEGER NOT NULL REFERENCES usuarios(id),  
  mensagem TEXT NOT NULL,  
  criado_em TEXT DEFAULT CURRENT_TIMESTAMP  
);
```

4. Funcionalidades por fases

Fase 1 — Fundação

- Setup do projeto (Flask + SQLite)
- Autenticação (registro, login, JWT)
- CRUD básico de usuários com papéis

Fase 2 — Núcleo do sistema

- CRUD de chamados (criar, listar, ver detalhe, editar status/prioridade)
- Categorias de chamado
- Regra de permissão: cliente só vê os próprios chamados

Fase 3 — Colaboração

- Comentários/interações no chamado
- Upload de anexos
- Histórico de mudanças de status

Fase 4 — Dashboard e usabilidade

- Endpoint de estatísticas (contadores por status/prioridade)
- Busca e filtros (status, prioridade, texto)
- Notificações simples

Fase 5 — Polimento

- Paginação nas listagens
- Testes automatizados das rotas principais
- Deploy (back e front)

5. Mockups de tela

Versão mais limpa e moderna do dashboard original enviado como referência.

BD

BlueDesk
Chamados de suporte

JB

Dashboard

Meus chamados

Novo chamado

Perfil

Total
12

Em andamento
5

Resolvidos
4

Abertos
3

Abertos

Impressora não imprime
Alta
Júlia Barbosa

Internet instável
Média
Maria Silva

Em andamento

VPN não conecta
Média
Carlos Eduardo

Sistema lento
Baixa
Rafael Almeida

Resolvidos

Acesso ao SoftExpert
Baixa
Fernanda Costa

Erro no Protheus
Baixa
João Pedro

Dashboard — visão geral dos chamados

Abrir novo chamado
Descreva o problema que você está enfrentando

Título

Descrição

Categoria

Prioridade

Abrir chamado

Aba "Novo chamado" — formulário de abertura

Meus chamados

Impressora não imprime

Aberto em 03/05/2026

Alta

Aberto

Erro ao acessar ERP

Aberto em 03/05/2026

Média

Em andamento

Acesso ao SoftExpert

Resolvido em 01/05/2026

Baixa

Resolvido

Aba “Meus chamados” — lista com busca e filtros

JB

Júlia Barbosa

Cliente

Nome

E-mail

Papel

Salvar alterações

Aba “Perfil” — dados do usuário logado

6. Versionamento com Git e GitHub

Configuração inicial

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email@exemplo.com"
```

Geração da chave SSH

```
ssh-keygen -t ed25519 -C "seu-email@exemplo.com"
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub    # cole no GitHub em Settings > SSH and GPG keys
ssh -T git@github.com        # testar conexão
```

Criar e subir o repositório

```
cd caminho/para/bluedesk
git init
git remote add origin git@github.com:seu-usuario/bluedesk.git
git add .
git commit -m "commit inicial"
git branch -M main
git push -u origin main
```

.gitignore recomendado

```
__pycache__/
*.pyc
.env
venv/
.vscode/
bluedesk.db
```

Fluxo do dia a dia no VS Code

- Painel Source Control (Ctrl+Shift+G) mostra os arquivos alterados
- Stage dos arquivos (+) e escreve a mensagem de commit
- Clica em Sync Changes (ou roda git push no terminal)

Para o push acontecer automaticamente após cada commit feito pelo VS Code, adicione em settings.json:

```
"git.postCommitCommand": "push"
```


7. Implementação das abas funcionais

Cada aba segue o mesmo ciclo: carregar dados ao abrir a página (GET) → usuário interage → enviar mudança (POST/PUT) → atualizar a tela.

api.js — wrapper compartilhado de requisições

```
const API_URL = "http://localhost:5000/api/v1";

async function apiFetch(endpoint, options = {}) {
  const token = localStorage.getItem("token");
  const res = await fetch(`${API_URL}${endpoint}`, {
    ...options,
    headers: {
      "Content-Type": "application/json",
      Authorization: `Bearer ${token}`,
      ...options.headers,
    },
  });
  if (!res.ok) throw new Error(await res.text());
  return res.json();
}
```

Novo chamado

```
@app.route("/api/v1/chamados", methods=["POST"])
@requer_login
def criar_chamado(usuario):
    dados = request.get_json()
    db.execute(
        """INSERT INTO chamados
           (titulo, descricao, categoria_id, prioridade, solicitante_id, status)
           VALUES (?, ?, ?, ?, ?, 'aberto')""",
        (dados["titulo"], dados["descricao"], dados["categoria_id"],
         dados["prioridade"], usuario["id"]),
    )
    db.commit()
    return jsonify({"mensagem": "Chamado criado"}), 201
```

Meus chamados

```
@app.route("/api/v1/chamados/me", methods=["GET"])
@requer_login
def meus_chamados(usuario):
    status = request.args.get("status")
    query = "SELECT * FROM chamados WHERE solicitante_id = ?"
    params = [usuario["id"]]
    if status:
        query += " AND status = ?"
        params.append(status)
    query += " ORDER BY criado_em DESC"
    chamados = db.execute(query, params).fetchall()
    return jsonify([dict(c) for c in chamados])
```

Perfil

```

@app.route("/api/v1/usuarios/me", methods=["GET"])
@requer_login
def meu_perfil(usuario):
    return jsonify(usuario)

@app.route("/api/v1/usuarios/me", methods=["PUT"])
@requer_login
def atualizar_perfil(usuario):
    dados = request.get_json()
    db.execute(
        "UPDATE usuarios SET nome = ?, email = ? WHERE id = ?",
        (dados["nome"], dados["email"], usuario["id"]),
    )
    db.commit()
    return jsonify({"mensagem": "Perfil atualizado"})

```

Resumo do ciclo por aba

Aba	Método	Rota	O que faz
Novo chamado	POST	/chamados	cria registro vinculado ao usuário logado
Meus chamados	GET	/chamados/me	lista filtrada pelo solicitante_id
Perfil	GET / PUT	/usuarios/me	lê e atualiza os dados do próprio usuário